

AI Lab: Uniform Cost Search and Depth-Limited Search

Q1. Cost-Aware Delivery Robot using Uniform Cost Search (UCS)

Problem Statement

Design and implement a Python program for a delivery robot that must travel from a starting position to a destination in a two-dimensional grid. Different traversable cells have different travel costs. Use Uniform Cost Search (UCS) to find a route having the minimum total path cost.

Problem Representation

The delivery area is represented by a grid. The robot can move one cell at a time in four directions: up, down, left, or right. The robot cannot cross blocked cells. The cost of entering a traversable cell depends on the type of terrain.

Symbol	Meaning	Traversal Cost
S	Starting position of the robot	0
G	Destination / goal	1
R	Normal road	1
M	Moderate-cost road / rough surface	3
T	High-cost road / traffic-congested cell	5
X	Blocked area / obstacle	Not traversable

Uniform Cost Search Requirement

UCS must always expand the frontier node having the smallest accumulated path cost $g(n)$. Use a priority queue (min-heap) ordered by $g(n)$. If a cheaper path to an already discovered state is found, update its cost and parent information.

- Start with S having path cost $g(S) = 0$.
- Generate all valid neighbouring cells and compute their new accumulated costs.
- Insert or update frontier entries according to the minimum $g(n)$ value.
- Continue until G is removed from the priority queue for expansion.
- Reconstruct the minimum-cost path using parent pointers.

Important Observation

UCS minimizes total path cost, not the number of movements. Therefore, it may select a route with more steps if that route avoids expensive terrain and has a lower total cost.

Example Grid

S	R	R	T	T	R	R	R	R	R
X	X	R	T	X	R	X	X	X	R
R	R	R	T	R	R	R	R	X	R
R	X	X	T	R	X	X	R	X	R
R	R	R	R	R	R	X	R	R	R
R	X	X	X	X	R	X	X	X	R
R	R	R	R	X	R	R	R	R	R

X	X	X	R	X	X	X	X	X	R
R	R	R	R	R	R	R	R	R	R
R	X	X	X	X	X	X	X	X	G

Expected Output (For the above example)

--- Uniform Cost Search ---

Start: (0,0)

Goal: (9,9)

Minimum-cost route:

(0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (2,3) -> (2,4) -> (2,5) -> (2,6) -> (2,7) -> (3,7) -> (4,7) -> (4,8)
-> (4,9) -> (5,9) -> (6,9) -> (7,9) -> (8,9) -> (9,9)

Total movements: 18

Total path cost: 22

Implementation Requirements

- Accept a weighted grid of size at least 10 x 10 as input.
- The grid must contain one start S, one goal G, traversable cells R/M/T, and blocked cells X.
- Implement UCS using a priority queue (heapq in Python may be used).
- Maintain the accumulated path cost $g(n)$ for every discovered state.
- Maintain parent information so that the final route can be reconstructed.
- Display the order in which nodes are expanded, the final route, total movements, and total path cost.
- The program should report 'No path exists' if the destination is unreachable.
- Do not use BFS, DFS, DLS, A*, or any library shortest-path routine in place of UCS.

Q2. Rescue Robot using Depth-Limited Search (DLS)

Problem Statement

Design and implement a Python program for a rescue robot operating in a two-dimensional grid. The robot must search for a target location using Depth-Limited Search (DLS). The purpose of the assignment is to study how imposing a depth limit prevents unrestricted deep exploration, while also showing that an insufficient limit can cause the search to miss an existing solution.

Problem Representation

The rescue area is represented by a grid. The robot starts at S and must reach G. It can move one cell at a time in four directions and cannot move through blocked cells. The depth of a node is the number of moves from the starting state.

Symbol	Meaning
S	Starting position of the robot
G	Goal / rescue target
R	Traversable cell
X	Blocked area / obstacle

Depth-Limited Search Requirement

DLS behaves like depth-first search, but it does not expand a node beyond a user-specified depth limit L . Use the fixed successor order Right, Down, Left, Up whenever more than one move is possible. Maintain the current search path to prevent cycles.

- Depth of S is 0.
- If the current node is G , return SUCCESS and reconstruct the path.
- If the current depth equals L and the node is not G , return CUTOFF for that branch.
- Otherwise, recursively explore valid successors in the specified order.
- If every branch within the limit is exhausted without reaching G , report FAILURE/CUTOFF as appropriate.

Two Required Runs

Run the same grid with at least two different depth limits so that students can directly observe the effect of the limit.

- Run 1: use $L = 18$. The search should terminate without finding the goal because the solution lies deeper than the limit.
- Run 2: use $L = 25$. The search should be able to reach the goal in the given example.

Example Grid

S	R	R	X	R	R	R	R	R	R
X	X	R	X	R	X	X	X	X	R
R	R	R	X	R	R	R	R	X	R
R	X	X	X	X	X	X	R	X	R
R	R	R	R	R	R	X	R	R	R
X	X	X	X	X	R	X	X	X	R
R	R	R	R	X	R	R	R	R	R
R	X	X	R	X	X	X	X	X	R
R	R	R	R	R	R	R	R	R	R
X	X	X	X	X	X	X	X	X	G

Expected Output (For the above example)

--- Depth-Limited Search: Limit = 18 ---

Result: CUTOFF / Goal not found within depth limit

Maximum allowed depth: 18

--- Depth-Limited Search: Limit = 25 ---

Result: SUCCESS

Route:

(0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (2,1) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)
-> (4,5) -> (5,5) -> (6,5) -> (6,6) -> (6,7) -> (6,8) -> (6,9) -> (7,9) -> (8,9) -> (9,9)

Solution depth / movements: 22

Implementation Requirements

- Accept a grid of size at least 10 x 10 and a depth limit L as input.
- Implement DLS explicitly using recursion or an explicit stack; do not call a ready-made graph-search function.
- Use the successor order Right, Down, Left, Up for reproducible results.
- Prevent immediate cycles by ensuring that a state already present on the current search path is not revisited on that path.
- Display the sequence of expanded states, the maximum depth reached, and the final result.
- Clearly distinguish SUCCESS, CUTOFF, and FAILURE in the program output.
- If SUCCESS is returned, display the complete route and number of movements.
- Run the program for at least two different depth limits and briefly explain why the outcomes differ.

Comparison / Discussion Questions

- Why can DLS fail to find a goal even when a valid path exists?
- How does increasing L affect time and memory usage?
- How is DLS different from ordinary DFS?
- Is DLS guaranteed to return the shortest path? Explain using your observations.